implement scheduler(CFS)

0x00. requirement

> ALL DONE BY MYSELF w/o any LLM!!!!

0x01. scheduling policy

selection

- select the RUNNABLE process w/ lowest 'virtual runtime'
- Each process has its own virtual runtime. The scheduler updates the running process's virtual runtime based on its actual runtime and its priority.

scheduler data structure

- use Red-Black tree
- insertion, deletion, lookup 모두에 $O(\log n)$ 이므로

0x02. CFS

virtual runtime

> A metric that tracks the total CPU execution time a process has been scheduled, scaled by its priority

- nice value = A priority hint that determines how rapidly virtual runtime is accumulated.
  - A higher nice value causes the scheduler to increase the virtual runtime value at a faster rate. As a result, the process is selected for execution less frequently.
  - The nice value must always be in the range from -3 to 2.
    - Initialize the nice value to 0 (default priority) for the very first process.
    - When a process is forked, it inherits its parent's nice value.

rule

1. Every time a process is selected, its virtual runtime is increased based on its priority.
2. Each increment of 1 in the nice value doubles the amount of virtual runtime added.
3. When a process is forked, initialize its virtual runtime to its parent's virtual runtime. If there is no parent, initialize it to 0.
4. When a process wakes up from the SLEEPING, update its virtual runtime to the minimum virtual runtime among all RUNNABLE processes.

1. RUNNABLE 프로세스가 존재하는 경우
→ 명세대로 RUNNABLE 프로세스들 중 최소 virtual runtime으로 갱신
2. RUNNABLE 프로세스는 없지만, RUNNING 중인 프로세스가 하나 이상 존재하는 경우
→ RUNNING 프로세스들의 virtual runtime 중 최솟값으로 갱신
3. RUNNABLE도 RUNNING도 존재하지 않는 경우
→ virtual runtime을 0으로 설정

w/ set_nice system call

`int set_nice(int nice)`

- nice (int): The target nice value to apply. Must be within the valid range (from -3 to 2).
- Return value:
  - Return 0 if the nice value is successfully updated.
  - Return 1 if the provided nice value is invalid.

0x10. DESIGN

0x11. scheduler design

0. policy

- vruntime += tick * f(nice)

nice	증가량
-3	1
-2	2
-1	4
0	8
1	16
2	32

- RUNNABLE이 되면 → tree에 insert
- scheduler가 선택해서 RUNNING이 되면 → tree에서 delete
- SLEEPING, ZOMBIE, UNUSED면 → tree에 없어야 함

1. define RB tree for each 'Run Queue'

> trouble shooting 참고: implement scheduler(CFS) > 0x33. concurrency of scheduling

- 각 CPU는 자기 rq만 접근→ single-writer model
  - `struct rq run_queues[MAX_CPU];`
    - global -> kernel data area
  - 동일한 DS를 여러 CPU가 동시에 수정하지 않음 → lock contention 제거
    1. CPU_i는 rq_i에서 task 선택
    2. vruntime update → rq_i 내부에서만 수행
    3. enqueue/dequeue → local operation
- run_queue의 정의

```
struct run_queue {
    int  cpu_idx;
    spinlock_t lock; // 내부 contention 방지용
    struct rb_root timeline;
    struct rb_node *leftmost; // 이제 실행할 process
    int nr_running; // 현재 rq에 들어 있는 runnable task 수
    struct task *curr; // 지금 이 CPU에서 실제로 실행 중인 task
};
```

2. CPU distribution policy

> locality를 최대한 유지하면서, 필요할 때만 최소한으로 이동한다

- 1순위: last_cpu
- 2순위: idle CPU
- 3순위: least-loaded CPU
- 단, 차이가 작으면 그냥 last_cpu

0x12. scheduling 필요한 순간들

0. fork

- 언제:
  - fork() 완료 후
  - 초기 process 생성 후
- 동작:
  - p->state = RUNNABLE
  - child->nice = parent->nice
  - if no parent (== init process???)
    - set_nice(0)
  - child->vruntime = parent->vruntime

1. timer interrupt (preemption)

> 일정 시간 slice 끝남;
> 강제 context switch 가능

- RUNNING → RUNNABLE 상태 변경
- 해당 cpu가 관리하던 run queue에서 dequeue

2. blocking event

> process(thread)가 더 못 진행할 때 i.e., RUNNING → SLEEPING

- I/O 요청 후 대기
- lock 대기
- sleep()

3. wakeup event

> 대기하던 process(thread)가 다시 runnable

- I/O 완료 interrupt
- semaphore signal

4. system call

- 일부 syscall에서 scheduling 발생
- yield()
- sleep()
- read() (blocking)

5. exception / fault

> page fault 처리 중 block 가능

0x20. IMPL

0x21. scheduling Data Structure

1. cpu별로 run_queue를 관리
2. run_queue는 Red-Black tree를 담고 있음
   - 각 노드는 vruntime을 key로 가지고 있고
   - data에는 proc* 타입을 담고 있음.

```
// proc.h
struct cpu {
  struct proc *proc;
  struct context context;
  int noff;
  int intena;
  struct run_queue *rq;
  int online;
};

struct run_queue {
    struct spinlock lock;
    struct rb_tree *tree;
    struct rb_node *leftmost;
    uint64 nr_running;
    uint64 min_vruntime;
};

struct proc {
    struct spinlock lock;

    enum procstate state;
    void *chan;
    int killed;
```

```
    int xstate;
    int pid;

    struct proc *parent;

    uint64 kstack;
    uint64 sz;
    pagetable_t pagetable;
    struct trapframe *trapframe;
    struct context context;
    struct file *ofile[NOFILE];
    struct inode *cwd;
    char name[16];

    uint64 last_scheduled_time;
    int nice_value;
    uint64 vruntime;

    struct rb_node run_node;
    struct cpu *home_cpu;
    struct run_queue *on_rq;

    int queued;
};

// rbtree.h
struct rb_node {
  uint64 key; // vruntime
  struct proc* data; // proc *
  struct rb_node *parent;
  struct rb_node *left;
  struct rb_node *right;
  rb_color color;
};
```

이후 정보를 어떻게 담는가

> main() → procinit() → 각 cpu.rq 초기화 → userinit()에서 첫 프로세스 생성/enqueue → 각 CPU scheduler() 실행

```
// procinit()
void
procinit(void)
{
  struct proc *p;
  struct cpu *c;

  initlock(&pid_lock, "nextpid");
  initlock(&wait_lock, "wait_lock");

  for(c = cpus; c < &cpus[NCPU]; c++) {
    c->rq = &run_queues[c - cpus];
    initlock(&c->rq->lock, "runqueue");
    c->rq->tree = &run_queue_trees[c - cpus];
    c->rq->tree->root = 0;
    c->rq->leftmost = 0;
    c->rq->nr_running = 0;
  }

  for(p = proc; p < &proc[NPROC]; p++) {
      initlock(&p->lock, "proc");
      p->state = UNUSED;
      p->kstack = KSTACK((int) (p - proc));
  }
}

// userinit()
void
userinit(void)
{
```

```
  struct proc *p;

  p = allocproc();
  initproc = p;

  p->cwd = namei("/");

  p->state = RUNNABLE;
  enqueue(p);

  release(&p->lock);
}
```

0x22. dynamics — run queue operations

1. enqueue w/ selecting new cpu

```
void
enqueue_on_rq(struct run_queue *rq, struct proc *p, enum procstate from)
{
  if(p->queued)
    panic("double enqueue");

  p->vruntime = compute_enqueue_vruntime(rq, p, from);
  p->run_node.key = p->vruntime;
  p->run_node.data = p;

  rb_insert(rq->tree, &p->run_node);
  rq->leftmost = rb_first(rq->tree->root);
  if(rq->leftmost)
    rq->min_vruntime = rq->leftmost->key;
  else
    rq->min_vruntime = p->vruntime;
  rq->nr_running++;

  p->state = RUNNABLE;
  p->queued = 1;
  p->on_rq = rq;
}
```

logic

- 새로 할당한 cpu's run queue에 할당
- proc 정보
  - vruntime 계산 (아래 helper참고)
  - run_queue INSERTION

```
uint64
compute_enqueue_vruntime(struct run_queue *nrq, struct proc *p, enum procstate from)
{
  uint64 vruntime;
  uint64 min;

  switch (from) {
    case USED: {
      struct proc *pp = p->parent;
      if (pp) {
        vruntime = pp->vruntime;
      } else {
        vruntime = 0; // init
      }
      break;
    }
    case SLEEPING: {
      if (find_global_minimum_vruntime(nrq, &min)) {// 1, RUNNABLE 프로세스가 존재하는 경우
        vruntime = min;
      } else if (find_minimum_running_vruntime(&min)) { // 2. RUNNABLE 프로세스는 없지만, RUNNING 중인 프로세스가 하나
```

```
이상 존재하는 경우
        vruntime = min;
      } else // 3. RUNNABLE도 RUNNING도 존재하지 않는 경우
          vruntime = 0;
      break;
    }
    default: {
      struct run_queue *rq = mycpu()->rq;
      if (rq != nrq) {
        acquire(&rq->lock);
        shift_scale_vruntime(mycpu()->rq, nrq, p);
        release(&rq->lock);
      }
      vruntime = p->vruntime + calculate_vruntime_delta(p);
      break;
    }
  }
  return vruntime;
}

int
find_minimum_running_vruntime(uint64 *out)
{
  struct cpu *c;
  uint64 min = 0;
  int found = 0;

  for(c = cpus; c < &cpus[NCPU]; c++) {
    if(!c->online)
      continue;


    if (c == mycpu()) {
      min = c->rq->min_vruntime;
      continue;
    }


    if(c->proc != 0) {
      struct proc *p = c->proc;
      uint64 cur = p->vruntime + calculate_vruntime_delta(p);

      if(!found || cur < min)
        min = cur;
      found = 1;
    }
  }

  if(found){
    *out = min;
    return 1;
  }
  return 0;
}

int
find_global_minimum_vruntime(struct run_queue *held_rq, uint64 *out)
{
  struct cpu *c;
  uint64 min = 0;
  int found = 0;

  for(c = cpus; c < &cpus[NCPU]; c++) {
    struct run_queue *rq = c->rq;
    uint64 tmp;

    if(!c->online)
      continue;

    if(rq != held_rq)
```

```
      acquire(&rq->lock);

    if(rq->nr_running > 0) {
      tmp = rq->min_vruntime;
      if(!found || tmp < min)
        min = tmp;
      found = 1;
    }

    if(rq != held_rq)
      release(&rq->lock);
  }

  if(found) {
    *out = min;
    return 1;
  }
  return 0;
}

uint64 // must need mutual exclusion
calculate_vruntime_delta(struct proc *p)
{
  uint64 runtime = r_time() - p->last_scheduled_time;
  return nice_to_vruntime_step[nice_to_index(p->nice_value)] * runtime;
}

void
shift_scale_vruntime(struct run_queue *src_rq, struct run_queue *dst_rq, struct proc *p)
{
  uint64 delta = 0;

  if(p->vruntime > src_rq->min_vruntime)
    delta = p->vruntime - src_rq->min_vruntime;

  p->vruntime = dst_rq->min_vruntime + delta;
}
```

vruntime의 계산

- USED = fork/init을 위해 새로 생성된 프로세스는 다음 로직을 따름
  - 명세: When a process is forked, initialize its virtual runtime to its parent's virtual runtime. If there is no parent, initialize it to 0.
- SLEEPING = IO 등을 이유로 block된 경우
  - IF RUNNABLE 있음 -> 전체 cpu중 vruntime 최저치로
  - ELSE IF RUNNING 중 vruntime 최저
  - ELSE -> 0
- 그 외
  - 새로운 cpu로 옮겼을 경우 -> 증가 정도가 다르기 때문에 scale 차이를 반영해 업데이트

selecting new cpu

```
struct cpu*
select_target_cpu(struct proc *p)
{
  // 1순위: last cpu
  // 2순위: idle cpu
  // 3순위: least-loaded cpu
  struct cpu *c;
  struct cpu *last = 0;
  struct cpu *idle = 0;
  struct cpu *least = 0;

  int last_load = 0;
  int least_load = 0x7fffffff;

  for(c = cpus; c < &cpus[NCPU]; c++){
    if(!c->online)
      continue;
```

```
    if(c == p->home_cpu){
      last = c;
      last_load = c->rq->nr_running;
      continue;
    }

    acquire(&c->rq->lock);
    int load = c->rq->nr_running;
    int is_idle = (c->proc == 0 && load == 0);

    if(is_idle && idle == 0)
      idle = c;

    if(least == 0 || load < least_load){
      least = c;
      least_load = load;
    }

    release(&c->rq->lock);
  }

  if(last){
    if(least == 0)
      return last;

    if(last_load - least_load <= LOAD_DIFF_THRESHOLD)
      return last;
  }

  if(idle)
    return idle;

  if(least)
    return least;

  return mycpu();
}
```

- 최근에 사용한 cpu
- 아니면 놀고 있는 Idle
- 아니면 가장 적게 load된 cpu를
- 반환

2. scheduler w/ dequeueing

```
void
scheduler(void)
{
  // 한 cpu가 전제되어 있음 항상. 각 코어별로 main()에서 실행된 후 looping!
  struct cpu *c = mycpu();
  struct run_queue *rq;

  while (1) { //scheduling round

    intr_on();
    intr_off();

    rq = c->rq;
    acquire(&rq->lock);

    if (rq == 0) {
      release(&rq->lock);
      continue;
    }

    if (rq->nr_running == 0 || rq->leftmost == 0) {
      release(&rq->lock);
```

```
      asm volatile("wfi");
      continue;
    }

    struct proc *p = rq->leftmost->data; // 다음에 실행할거
    int found = 0;

    acquire(&p->lock);
    if(p->state == RUNNABLE) {
      p->state = RUNNING;
      p->home_cpu = c;
      c->proc = p;
      p->last_scheduled_time = r_time();

      dequeue_from_rq(rq, p);
      release(&rq->lock);
      swtch(&c->context, &p->context);

      c->proc = 0;
      found = 1;
    } else {
      dequeue_from_rq(rq, p);
      release(&rq->lock);
      found = 1;
    }
    release(&p->lock);

    if(found == 0) {
      asm volatile("wfi"); // CPU idle loop 용. WFI = Wait For Interrupt.
    }
  }
}
```

dequeue

```
void
dequeue_from_rq(struct run_queue *rq, struct proc *p)
{
  if(!p->queued || p->on_rq != rq)
    panic("bad dequeue");

  rb_delete(rq->tree, &p->run_node);
  rq->nr_running--;
  rq->leftmost = rb_first(rq->tree->root);
  if(rq->leftmost)
    rq->min_vruntime = rq->leftmost->key;
  else
    rq->min_vruntime = p->vruntime;

  p->queued = 0;
  p->on_rq = 0;
}
```

0x23. init

```
// Set up first user process.
void
userinit(void)
{
  struct proc *p;
  struct run_queue *rq = mycpu()->rq;

  acquire(&rq->lock);
  p = allocproc();
  initproc = p;

  p->cwd = namei("/");
```

```
  enqueue_on_rq(rq, p, p->state);

  release(&p->lock);
  release(&rq->lock);
}

static struct proc*
allocproc(void)
{
  struct proc *p;

  for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if(p->state == UNUSED) {
      goto found;
    } else {
      release(&p->lock);
    }
  }
  return 0;

found:
  p->pid = allocpid(); // 1. allocate a new PID
  p->state = USED;  // 2. changes its state to USED
  p->nice_value = 0;
  p->home_cpu = 0;
  p->vruntime = 0;
  p->on_rq = 0;
  p->queued = 0;

  // Allocate a trapframe page.
  if((p->trapframe = (struct trapframe *)kalloc()) == 0){ // 3. allocates a trapframe page
    freeproc(p);
    release(&p->lock);
    return 0;
  }

  // An empty user page table.
  p->pagetable = proc_pagetable(p); // 4. allocates an empty user page table
  if(p->pagetable == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
  }

  // Set up new context to start executing at forkret,
  // which returns to user space.
  memset(&p->context, 0, sizeof(p->context)); // 5. initializes the process context
  p->context.ra = (uint64)forkret; // 6. set the return address to forkret()
  p->context.sp = p->kstack + PGSIZE;

  return p;
}
```

0x24. timer interrupt

```
uint64
usertrap(void)
{
  int which_dev = 0;

  if((r_sstatus() & SSTATUS_SPP) != 0)
    panic("usertrap: not from user mode");

  w_stvec((uint64)kernelvec);  //DOC: kernelvec

  struct proc *p = myproc();

  p->trapframe->epc = r_sepc();
```

```
  if(r_scause() == 8){

    if(killed(p))
      kexit(-1);

    p->trapframe->epc += 4;

    intr_on();

    syscall();
  } else if((which_dev = devintr()) != 0){
    // ok
  } else if((r_scause() == 15 || r_scause() == 13) &&
            vmfault(p->pagetable, r_stval(), (r_scause() == 13)? 1 : 0) != 0) {
    // page fault on lazily-allocated page
  } else {
    printf("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(), p->pid);
    printf("            sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
    setkilled(p);
  }

  if(killed(p))
    kexit(-1);

  // give up the CPU if this is a timer interrupt.
  if(which_dev == 2)
    yield();

  prepare_return();

  // the user page table to switch to, for trampoline.S
  uint64 satp = MAKE_SATP(p->pagetable);

  // return to trampoline.S; satp value in a0.
  return satp;
}

void
kerneltrap()
{
  int which_dev = 0;
  uint64 sepc = r_sepc();
  uint64 sstatus = r_sstatus();
  uint64 scause = r_scause();

  if((sstatus & SSTATUS_SPP) == 0)
    panic("kerneltrap: not from supervisor mode");
  if(intr_get() != 0)
    panic("kerneltrap: interrupts enabled");

  if((which_dev = devintr()) == 0){
    // interrupt or trap from an unknown source
    printf("scause=0x%lx sepc=0x%lx stval=0x%lx\n", scause, r_sepc(), r_stval());
    panic("kerneltrap");
  }

  // give up the CPU if this is a timer interrupt.
  if(which_dev == 2 && myproc() != 0)
    yield();

  // the yield() may have caused some traps to occur,
  // so restore trap registers for use by kernelvec.S's sepc instruction.
  w_sepc(sepc);
  w_sstatus(sstatus);
}
```

sched and yield

```
void
sched(void)
{
  int intena;
  struct proc *p = myproc();

  if(!holding(&p->lock))
    panic("sched p->lock");
  if(mycpu()->noff != 1)
    panic("sched locks");
  if(p->state == RUNNING)
    panic("sched RUNNING");
  if(intr_get())
    panic("sched interruptible");

  intena = mycpu()->intena;
  swtch(&p->context, &mycpu()->context);
  mycpu()->intena = intena;
}

// Give up the CPU for one scheduling round.
void
yield(void)
{
  struct proc *p = myproc();
  struct run_queue *rq = mycpu()->rq;
  acquire(&rq->lock);
  acquire(&p->lock);
  enqueue_on_rq(rq, p, RUNNING);
  release(&rq->lock);
  sched();
  release(&p->lock);
}
```

0x25. fork

```
// Create a new process, copying the parent.
// Sets up child kernel stack to return as if from fork() system call.
int
kfork(void)
{
  int i, pid;
  struct proc *np;
  struct proc *p = myproc();
  struct cpu *nc;

  //ㄱ. Allocate process.
  if((np = allocproc()) == 0){
    return -1;
  }

  // ㄴ. Copy user memory from parent to child.
  // 1. page table 새로 형성
  // 2. Physical memory에 데이터 복사
  // 3. 부모 page table 복사: 동일 va위에 복사된 data reference
  if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
    freeproc(np);
    release(&np->lock);
    return -1;
  }
  np->sz = p->sz;

  // ㄷ. copies parent's TRAPFRAME
  *(np->trapframe) = *(p->trapframe);

  // ㄹ. sets a0 register of the child's TRAPFRAME to 0
  np->trapframe->a0 = 0;
```

```
  // ㅁ. copies parent's file descriptors and current working directory
  for(i = 0; i < NOFILE; i++)
    if(p->ofile[i])
      np->ofile[i] = filedup(p->ofile[i]);
  np->cwd = idup(p->cwd);

  //ㅂ. copies parent's process name
  safestrcpy(np->name, p->name, sizeof(p->name));

  pid = np->pid;

  release(&np->lock);

  // ㅅ. sets the parent of the newly created process to the on that called fork()
  acquire(&wait_lock);
  np->parent = p;
  release(&wait_lock);

  // ㅇ. sets new process's state to RUNNABLE so that child process is ready to be scheduled
  nc = select_target_cpu(np);

  acquire(&nc->rq->lock);
  acquire(&np->lock);
  np->nice_value = p->nice_value;
  np->home_cpu = nc;
  enqueue_on_rq(nc->rq, np, np->state);
  release(&np->lock);
  release(&nc->rq->lock);
  return pid;
}
```

0x26. sleep

> may be by IO interrupt
> -> removed from run queue

```
void
sleep(void *chan, struct spinlock *lk)
{
  struct proc *p = myproc();

  acquire(&p->lock);  //DOC: sleeplock1
  release(lk);

  // Go to sleep.
  p->chan = chan;
  p->state = SLEEPING;
  p->vruntime += calculate_vruntime_delta(p);

  sched();

  // Tidy up.
  p->chan = 0;

  // Reacquire original lock.
  release(&p->lock);
  acquire(lk);
}
```

0x27. kill & wakeup

kill

```
int
kkill(int pid)
{
  struct proc *p;
  struct cpu *nc;
```

```
  // ㄱ. scan the entire process table and finds the process with the matching PID
  for(p = proc; p < &proc[NPROC]; p++){
    nc = select_target_cpu(p);
    acquire(&nc->rq->lock);
    acquire(&p->lock);
    if(p->pid == pid){
      p->killed = 1;

      // if found, it sets the process' killed field to 1 and changes its state to RUNNABLE to wake it up if
it is sleeping
      if(p->state == SLEEPING){
        // Wake process from sleep().
        p->state = RUNNABLE;
        enqueue_on_rq(nc->rq, p, SLEEPING);
      }
      release(&p->lock);
      release(&nc->rq->lock);
      return 0;
    }
    release(&p->lock);
    release(&nc->rq->lock);
  }
  return -1;
}
```

wakeup

```
void
wakeup(void *chan)
{
  struct proc *p;
  struct cpu *nc;

  for(p = proc; p < &proc[NPROC]; p++) {
    nc = select_target_cpu(p);
    if(p != myproc()){
      acquire(&nc->rq->lock);
      acquire(&p->lock);
      if(p->state == SLEEPING && p->chan == chan) {
        enqueue_on_rq(nc->rq, p, p->state);
      }
      release(&p->lock);
      release(&nc->rq->lock);
    }
  }
}
```

0x30. PS; Troubleshooting

> w/ 궁금증 해결

0x31. nice value 정보는 어떻게 관리해야 하는가?

- nice value의 정보를 process에 귀속시켜서 관리해야 하는데 이건 어떡하는가?

idea1. MLFQ; Multi-Level Feedback Queue의 queue 구분 기준으로 nice value를 사용?

- promotion, demotion 시 속한 queue정보에 따라 vruntime도 수정 가능하도록?

기각 사유

1. “왜 이 queue에 들어왔는가”와 “지금 어느 queue에 있는가”가 구분되지 않는다.
2. vruntime은 CFS류 개념이고, promotion/demotion queue는 MLFQ류 개념이다. 이 둘을 혼용하려면 복잡성을 감안해야 함.

idea2. 별도의 pid별 nice value를 다루는 딕셔너리 자료 구조를 관리

기각 사유

1. process 생성, fork, exit, reuse 시점마다 별도 테이블과 동기화를 맞춰야 한다
2. scheduling마다, pid -> table lookup -> nice를 거치면, 구조가 괜히 무거워진다.

idea3. PCB에서 관리

- proc struct에 priority (nice value) 정보 추가

궁금증: nice value는 안바뀌나?

- 자동으로 바뀌지 않음
- process 생성 시 inherit말고는

0x32. scheduling 방법

- 각 CPU
  - CPU->context 정보 (global)
  - 평생 cpu->context에는 scheduler()의 특정 순간의 register set이 저장되는 것.
  - 이게 다시 scheduler용 kernel stack에서 다시 실행됨; scheduler는 cpu별 kernel stack 위에서 실행되고, context는 그 stack의 resume 지점(register snapshot)일 뿐

p1->scheduler

> 현재 cpu에 저장된 scheduler register context가 불러와짐.

```
void
yield(void)
{
struct proc *p = myproc();
acquire(&p->lock);
p->state = RUNNABLE;
sched();
release(&p->lock);
}

void
sched(void)
{
    int intena;
    struct proc *p = myproc();
    if(!holding(&p->lock))
        panic("sched p->lock");
    if(mycpu()->noff != 1)
        panic("sched locks");
    if(p->state == RUNNING)
        panic("sched RUNNING");
    if(intr_get())
        panic("sched interruptible");
    intena = mycpu()->intena;
    swtch(&p->context, &mycpu()->context);
    mycpu()->intena = intena;
}
```

- &p->context: 현재 실행 중인 kernel thread의 register를 proc 구조체 내부 메모리(global)에 저장됨.
- &mycpu()->context: cpu->context에 저장된 register를 load해서 scheduler 실행을 resume함

scheduler->p2

`swtch(&c->context, &p->context);`

- &c->context: 현재 scheduler의 register가 cpu->context에 저장되고,
- &p->context: p->context를 load하면서 해당 process의 kernel stack으로 점프함

0x33. concurrency of scheduling

> 각 프로세스가 scheduler정보를 읽고 쓰는 접근에 concurrency를 고려해야 함.

idea1. memory 어디에 저장?

- scheduling에서 관리해야 할 정보
  - globally at heap
  - DS: using RB tree
- RB tree에 대한 operation에 대해 synchronization을 적용 (여기서는 spinlock)
  - 단순히 RB tree 정보만 보고 이를 기반으로 CPU별로 실행하는 흐름.

PS

1. 하지만 매우 빈번하게 cpu scheduling이 일어날텐데 그럼 lock이 매번 걸리면 lock contention이 매우 급증
   - CPU 수 = N → lock 경쟁 = O(N)
   - 각 CPU가 독립적으로 scheduling하려 해도 lock 때문에 직렬화됨
2. 문제 2: busy waiting 비용
   - spinlock → lock 못 잡으면 계속 spinning
   - scheduler path에서 CPU cycle 낭비 → 전체 throughput 감소

idea2. contention은 어떻게 관리할 것인가?

- 각 cpu별로 격리시킨 rb tree를 만들까?
  - runqueue를 CPU마다 분리
    - CPU0 → rq0
    - CPU1 → rq1
    - …
  - 각 rq는 RB tree 유지
- 각 CPU는 자기 rq만 접근→ single-writer model
  - 동일한 DS를 여러 CPU가 동시에 수정하지 않음 → lock contention 제거

1. CPU_i는 rq_i에서 task 선택
2. vruntime update → rq_i 내부에서만 수행
3. enqueue/dequeue → local operation

결론

1. 일단 run_queue를 kernel data영역에 할당
   `struct rq run_queues[MAX_CPU];`
2. run_queue의 정의

```
struct run_queue {
    int cpu_id;
    spinlock_t lock; // 내부 contention 방지용
    struct rb_root timeline;
    struct rb_node *leftmost; // 이제 실행할 process
    int nr_running; // 현재 rq에 들어 있는 runnable task 수
    struct task *curr; // 지금 이 CPU에서 실제로 실행 중인 task
};
```

idea3. cpu 분배방식; run_queue의 정의

> locality를 최대한 유지하면서, 필요할 때만 최소한으로 이동한다

- 1순위: last_cpu
- 2순위: idle CPU
- 3순위: least-loaded CPU
- 단, 차이가 작으면 그냥 last_cpu

근거

1. CPU cache (L1/L2/L3)에 해당 task의 working set이 남아있음
   - code path
   - data (heap 일부)
   - 이 cache에 올라가 있음 → 같은 CPU에서 다시 실행하면 cache hit
2. 만약 idle CPU가 있다면 그곳으로 보냄 즉, tradeoff가 발생

- CPU cache를 이용할지
- cache miss를 감수하고 실행

3. 만약 모든 CPU가 busy라면, 가장 덜 바쁜 CPU 선택

idea4. 만약 cpu가 변경됐을 때 vruntime 맥락이 다른 것은 어떻게 고려?

> 사실 이부분은 큰 문제가 되고, 실제로 Linux의 경우에도 Normalization을 거침.

--> 0x35

0x34. deadlock 발생

양상

- `yield()`, `wakeup()`, `kkill()` 모두 `p->lock`을 잡은 상태에서 `enqueue()` 호출한다.
- `enqueue()` 안에서는 `acquire(&rq->lock)` 한다.
- 즉 lock 순서가: `p->lock -> rq->lock`
- 반면 scheduler는: `rq->lock -> p->lock`

> lock acquire, release가 flow에 따라 널리 흩어져 있다보니 기초적인 deadlock 발생(ABBA deadlock pattern)을 못막는 일이 발생함.

```
void
yield(void)
{
  struct proc *p = myproc();
  acquire(&p->lock);
  p->state = RUNNABLE;
  add_vruntime(p);
  enqueue(p, RUNNING);
  sched();
  release(&p->lock);
}

void
wakeup(void *chan)
{
  struct proc *p;

  for(p = proc; p < &proc[NPROC]; p++) {
    if(p != myproc()){
      acquire(&p->lock);
      if(p->state == SLEEPING && p->chan == chan) {
        enqueue(p, p->state);
      }
      release(&p->lock);
    }
}

int
kkill(int pid)
{
  struct proc *p;

  // ㄱ. scan the entire process table and finds the process with the matching PID
  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
    if(p->pid == pid){
      p->killed = 1;

      // if found, it sets the process' killed field to 1 and changes its state to RUNNABLE to wake it up if
it is sleeping
      if(p->state == SLEEPING){
        // Wake process from sleep().
        enqueue(p, p->state);
      }
      release(&p->lock);
      return 0;
    }
```

```
    release(&p->lock);
  }
  return -1;
}

void
enqueue(struct proc *p, enum procstate state)
{
  struct cpu *nc = select_target_cpu(p);
  struct run_queue *rq = nc->rq;
  uint64 vruntime;
  uint64 min;

  acquire(&rq->lock);

  switch (state) {
    case USED: {
      struct proc *pp = p->parent;
      if (pp) {
        vruntime = pp->vruntime;
      } else {
        vruntime = 0; // init
      }
      break;
    }
    case SLEEPING: {
      if (nc->rq->nr_running > 0) { // 1, RUNNABLE 프로세스가 존재하는 경우
        struct rb_node *first = rb_first(nc->rq->tree->root);
        vruntime = first->key;
      } else if ((min = find_minimum_running_vruntime()) != 0) { // 2. RUNNABLE 프로세스는 없지만, RUNNING 중인 프
로세스가 하나 이상 존재하는 경우
        vruntime = min;
      } else { // 3. RUNNABLE도 RUNNING도 존재하지 않는 경우
        vruntime = 0;
      }
      break;
    }
    default: {
      vruntime = p->vruntime;
      break;
    }
  }

  p->state = RUNNABLE;

  p->vruntime = vruntime;
  p->run_node.key = vruntime;
  p->run_node.data = p;

  nc->rq->nr_running++;
  rb_insert(rq->tree, &p->run_node);
  rq->leftmost = rb_first(rq->tree->root);
  release(&rq->lock);
}
```

해결

> rq->lock -> p->lock 통일

1. enqueue / dequeue 로직 내부에서는 lock을 잡지 않고, 외부에서 잡은 후 들어오기
   - 일관성 유지

0x35. cpu별 vruntime scale 차이?

문제 상황

- 다른 cpu로 변경하고 해당 vruntime이 유지되는 경우
- 상이한 cpu scale을 가지고 있으므로 변경이 필요함.

IDEA: scale용

```
struct run_queue {
  struct spinlock lock;
  struct rb_tree *tree;
  struct rb_node *leftmost;
  int nr_running;
  uint64 min_vruntime;
};
```

- 이때 왜 Max값은 없어도 되는가?
  - 수학적으로 보면 max값을 알아야 함. 분포에 대한 정보 파악을 위해서
  - 하지만 증가분은 결국 nice value로 결정되고 차이값만 보정하면 되기에, min value를 가져온다.

```
void
shift_scale_vruntime(struct run_queue *src_rq, struct run_queue *dst_rq, struct proc *p)
{
  uint64 delta = 0;

  if(p->vruntime > src_rq->min_vruntime)
    delta = p->vruntime - src_rq->min_vruntime;

  p->vruntime = dst_rq->min_vruntime + delta;
}
```

- 따라서 위처럼 shifting을 통해 해결!

0x36. deadlock 2

양상

> 자꾸 test4에서 멈춤

IDEA

- 지금 이전 코드에선 다음 cpu를 고를때 같은 cpu인 경우의 예외 처리를 안했음
- 동일 lock에 대한 접근으로 self-deadlock이 걸림

```
struct cpu*
select_target_cpu(struct proc *p)
{
  // 1순위: last cpu
  // 2순위: idle cpu
  // 3순위: least-loaded cpu
  struct cpu *c;
  struct cpu *last = 0;
  struct cpu *idle = 0;
  struct cpu *least = 0;

  int last_load = 0;
  int least_load = 0x7fffffff;

  for(c = cpus; c < &cpus[NCPU]; c++){
    acquire(&c->rq->lock);
    int load = c->rq->nr_running;

    int is_idle = (c->proc == 0 && load == 0);



    if(is_idle && idle == 0)
      idle = c;

    if(least == 0 || load < least_load){
      least = c;
      least_load = load;
    }
```

```
    release(&c->rq->lock);
  }

  if(last){
    if(least == 0)
      return last;

    if(last_load - least_load <= LOAD_DIFF_THRESHOLD)
      return last;
  }

  if(idle)
    return idle;

  return least;
}
```

해결

```
if(c == p->home_cpu){
    last = c;
    last_load = c->rq->nr_running;
    continue;
}
```

위와 같은 예외 처리 로직 추가로 해결!

0x37. test5 실패

test5 실패?

- `woken process is not left behind after wakeup` fail은:
  - 깨어난 proc가 runnable은 됐는데
  - wakeup 뒤 측정 구간에서 hog들보다 **너무 적게 일했다**는 뜻

원인

```
 case SLEEPING: {
      if (nrq->nr_running > 0) { // 1, RUNNABLE 프로세스가 존재하는 경우
        struct rb_node *first = rb_first(nrq->tree->root);
        vruntime = first->key;
      } else if ((min = find_minimum_running_vruntime()) != 0) { // 2. RUNNABLE 프로세스는 없지만, RUNNING 중인 프로세스가 하나 이상 존재하는 경우
        vruntime = min;
      } else { // 3. RUNNABLE도 RUNNING도 존재하지 않는 경우
        vruntime = 0;
      }
      break;
    }
```

- 1에서 global이 아닌 새로 할당한 nrq에 대한 최솟값을 사용

0x40. RESULT

0x41. test1. nice value system call test

```
void
test_set_nice_boundary(void)
{
  printf("\n=== Test 1: set_nice range and return value ===\n");

  check(set_nice(-3) == 0, "set_nice(-3) returns 0");
  check(set_nice(-2) == 0, "set_nice(-2) returns 0");
```

```
  check(set_nice(-1) == 0, "set_nice(-1) returns 0");
  check(set_nice(0) == 0, "set_nice(0) returns 0");
  check(set_nice(1) == 0, "set_nice(1) returns 0");
  check(set_nice(2) == 0, "set_nice(2) returns 0");

  check(set_nice(-4) == 1, "set_nice(-4) returns 1");
  check(set_nice(3) == 1, "set_nice(3) returns 1");
  check(set_nice(-100) == 1, "set_nice(-100) returns 1");
  check(set_nice(100) == 1, "set_nice(100) returns 1");

  set_nice(0);
}
```

result

```
xv6 kernel is booting
$ ./cfstest
============================
  CFS Scheduler Test Suite
  works with CPUS=1 and SMP
============================

=== Test 1: set_nice range and return value ===
[PASS] set_nice(-3) returns 0
[PASS] set_nice(-2) returns 0
[PASS] set_nice(-1) returns 0
[PASS] set_nice(0) returns 0
[PASS] set_nice(1) returns 0
[PASS] set_nice(2) returns 0
[PASS] set_nice(-4) returns 1
[PASS] set_nice(3) returns 1
[PASS] set_nice(-100) returns 1
[PASS] set_nice(100) returns 1
```

test2,3

```
void
test_initial_nice_and_basic_fork(void)
{
  printf("\n=== Test 2: default nice and basic fork ===\n");

  set_nice(0);

  int pid = fork();
  if(pid == 0){
    int r1 = set_nice(0);
    int r2 = set_nice(NICE_MIN);
    int r3 = set_nice(NICE_MAX);
    exit(r1 == 0 && r2 == 0 && r3 == 0 ? 0 : 1);
  }

  int status;
  wait(&status);
  check(status == 0, "new process accepts valid nice range");
  set_nice(0);
}

void
test_nice_inheritance_by_share(void)
{
  printf("\n=== Test 3: nice inheritance through fork ===\n");

  int start_pipe[2];
```

```
  int result_pipe[2];
  int ok = 1;
  int total_low = 0;
  int total_high = 0;
  int n = WORKERS_PER_CLASS * 2;
  struct report reports[WORKERS_PER_CLASS * 2];

  pipe(start_pipe);
  pipe(result_pipe);

  for(int i = 0; i < WORKERS_PER_CLASS; i++){
    set_nice(NICE_MIN);
    spawn_counter(start_pipe[0], result_pipe[1], TAG_LOW, NICE_MIN, 0);
  }

  for(int i = 0; i < WORKERS_PER_CLASS; i++){
    set_nice(NICE_MAX);
    spawn_counter(start_pipe[0], result_pipe[1], TAG_HIGH, NICE_MAX, 0);
  }

  set_nice(0);
  start_children(start_pipe[1], n, uptime() + SHARE_TICKS);
  read_reports(result_pipe[0], reports, n, &ok);
  wait_all(n, &ok);

  for(int i = 0; i < n; i++){
    if(!reports[i].ok)
      ok = 0;
    if(reports[i].tag == TAG_LOW)
      total_low += reports[i].count;
    if(reports[i].tag == TAG_HIGH)
      total_high += reports[i].count;
  }
  printf("inherited nice -3 work: %d\n", total_low);
  printf("inherited nice  2 work: %d\n", total_high);
  check(ok && total_low > total_high + total_high / 2,
        "children inherit parent nice value");

  close(start_pipe[0]);
  close(start_pipe[1]);
  close(result_pipe[0]);
  close(result_pipe[1]);
  set_nice(0);
}
```

result

```
=== Test 2: default nice and basic fork ===
[PASS] new process accepts valid nice range

=== Test 3: nice inheritance through fork ===
inherited nice -3 work: 159638
inherited nice  2 work: 7411
[PASS] children inherit parent nice value
```

0x43. test4: cfs priority weighting

```
void
test_priority_weighting(void)
{
  printf("\n=== Test 4: CFS priority weighting ===\n");

  int result_pipe[2];
```

```
  int ok = 1;
  int n = NICE_COUNT * PRIORITY_WORKERS_PER_CLASS;
  int totals[NICE_COUNT];
  struct report reports[NICE_COUNT * PRIORITY_WORKERS_PER_CLASS];
  int start_tick = uptime() + PRIORITY_SETUP_TICKS;
  int measure_tick = start_tick + PRIORITY_WARMUP_TICKS;
  int stop_tick = measure_tick + PRIORITY_RUN_TICKS;

  for(int i = 0; i < NICE_COUNT; i++)
    totals[i] = 0;

  pipe(result_pipe);
  set_nice(NICE_MIN);

  for(int round = 0; round < PRIORITY_WORKERS_PER_CLASS; round++){
    if(round % 2 == 0){
      for(int nice = NICE_MIN; nice <= NICE_MAX; nice++)
        spawn_priority_counter(result_pipe[1], nice, start_tick, measure_tick, stop_tick);
    } else {
      for(int nice = NICE_MAX; nice >= NICE_MIN; nice--)
        spawn_priority_counter(result_pipe[1], nice, start_tick, measure_tick, stop_tick);
    }
  }

  set_nice(0);
  read_reports(result_pipe[0], reports, n, &ok);
  wait_all(n, &ok);

  for(int i = 0; i < n; i++){
    if(!reports[i].ok)
      ok = 0;
    if(reports[i].nice >= NICE_MIN && reports[i].nice <= NICE_MAX)
      totals[reports[i].nice - NICE_MIN] += reports[i].count;
  }

  for(int i = 0; i < NICE_COUNT; i++)
    printf("nice %d work: %d\n", i + NICE_MIN, totals[i]);

  int nonzero = 1;
  for(int i = 0; i < NICE_COUNT; i++){
    if(totals[i] == 0)
      nonzero = 0;
  }

  int ordered = 1;
  for(int i = 0; i < NICE_COUNT - 1; i++){
    if(totals[i] <= totals[i + 1])
      ordered = 0;
  }

  check(ok && nonzero, "all nice classes enter the measured interval");
  check(ok && nonzero && ordered, "lower nice receives more CPU work");
  check(ok && totals[0] > totals[3] && totals[3] > totals[5],
        "vruntime increment grows as nice increases");

  close(result_pipe[0]);
  close(result_pipe[1]);
  set_nice(0);
}
```

result

```
=== Test 4: CFS priority weighting ===
nice -3 work: 393610
nice -2 work: 287791
nice -1 work: 120359
nice 0 work: 48792
nice 1 work: 37610
nice 2 work: 13751
[PASS] all nice classes enter the measured interval
[PASS] lower nice receives more CPU work
[PASS] vruntime increment grows as nice increases
```

0x44. test5

```
void
test_wakeup_vruntime(void)
{
  printf("\n=== Test 5: wakeup vruntime update ===\n");

  int start_pipe[2];
  int result_pipe[2];
  int ok = 1;
  int hog_total = 0;
  int wake_total = 0;
  int n = WORKERS_PER_CLASS + 1;
  struct report reports[WORKERS_PER_CLASS + 1];

  pipe(start_pipe);
  pipe(result_pipe);

  int pid = fork();
  if(pid == 0){
    int stop_tick;
    struct report r;

    r.tag = TAG_WAKE;
    r.nice = 0;
    r.count = 0;
    r.ok = 1;

    if(set_nice(0) != 0)
      r.ok = 0;

    count_until(uptime() + 8);
    pause(5);

    if(read(start_pipe[0], &stop_tick, sizeof(stop_tick)) != sizeof(stop_tick)){
      r.ok = 0;
    } else {
      r.count = count_until(stop_tick);
    }

    write(result_pipe[1], &r, sizeof(r));
    exit(r.ok ? 0 : 1);
  }

  for(int i = 0; i < WORKERS_PER_CLASS; i++)
    spawn_counter(start_pipe[0], result_pipe[1], TAG_HOG, 0, 1);

  pause(8);
  start_children(start_pipe[1], n, uptime() + WAKE_TICKS);
  read_reports(result_pipe[0], reports, n, &ok);
  wait_all(n, &ok);
```

```
  for(int i = 0; i < n; i++){
    if(!reports[i].ok)
      ok = 0;
    if(reports[i].tag == TAG_WAKE)
      wake_total += reports[i].count;
    if(reports[i].tag == TAG_HOG)
      hog_total += reports[i].count;
  }

  printf("woken process work: %d\n", wake_total);
  printf("runnable competitors total work: %d\n", hog_total);
  check(ok && wake_total > 0, "sleeping process wakes and runs");
  check(ok && wake_total * WORKERS_PER_CLASS >= hog_total / 2,
        "woken process is not left behind after wakeup");

  close(start_pipe[0]);
  close(start_pipe[1]);
  close(result_pipe[0]);
  close(result_pipe[1]);
  set_nice(0);
}
```

result

```
=== Test 5: wakeup vruntime update ===
woken process work: 11330
runnable competitors total work: 61902
[PASS] sleeping process wakes and runs
[PASS] woken process is not left behind after wakeup
```

0x45. test6

```
void
test_wakeup_vruntime_running_only(void)
{
  printf("\n=== Test 6: wakeup vruntime with only RUNNING competitor ===\n");

  int result_pipe[2];
  int ok = 1;
  int hog_total = 0;
  int wake_total = 0;
  int start_tick = uptime() + 5;
  int stop_tick = start_tick + WAKE_TICKS;
  struct report reports[2];

  pipe(result_pipe);

  int pid = fork();
  if(pid == 0){
    struct report r;

    r.tag = TAG_CASE2;
    r.nice = 0;
    r.count = 0;
    r.ok = 1;

    pause(5);
    r.count = count_until(stop_tick);

    write(result_pipe[1], &r, sizeof(r));
    exit(0);
  }
```

```
  pid = fork();
  if(pid == 0){
    struct report r;

    r.tag = TAG_HOG;
    r.nice = 0;
    r.count = count_until(stop_tick);
    r.ok = 1;

    write(result_pipe[1], &r, sizeof(r));
    exit(0);
  }

  read_reports(result_pipe[0], reports, 2, &ok);
  wait_all(2, &ok);

  for(int i = 0; i < 2; i++){
    if(!reports[i].ok)
      ok = 0;
    if(reports[i].tag == TAG_CASE2)
      wake_total += reports[i].count;
    if(reports[i].tag == TAG_HOG)
      hog_total += reports[i].count;
  }

  printf("running-only wake work: %d\n", wake_total);
  printf("single running competitor work: %d\n", hog_total);
  check(ok && wake_total > 0, "process wakes when only a RUNNING process exists");
  check(ok && wake_total < hog_total * 2,
        "woken process does not restart far before running minimum");

  close(result_pipe[0]);
  close(result_pipe[1]);
  set_nice(0);
}
```

result

```
=== Test 6: wakeup vruntime with only RUNNING competitor ===
running-only wake work: 26902
single running competitor work: 37545
[PASS] process wakes when only a RUNNING process exists
[PASS] woken process does not restart far before running minimum
```

0x46. test 7

```
void
test_wakeup_vruntime_empty_system(void)
{
  printf("\n=== Test 7: wakeup vruntime with no RUNNABLE or RUNNING peer ===\n");

  int result_pipe[2];
  int ok = 1;
  struct report r;

  pipe(result_pipe);

  int pid = fork();
  if(pid == 0){
    struct report child_report;

    child_report.tag = TAG_CASE3;
    child_report.nice = 0;
    child_report.count = 0;
```

```
    child_report.ok = 1;

    pause(3);
    child_report.count = count_until(uptime() + WAKE_TICKS);

    write(result_pipe[1], &child_report, sizeof(child_report));
    exit(0);
  }

  read_reports(result_pipe[0], &r, 1, &ok);
  wait_all(1, &ok);

  if(!r.ok)
    ok = 0;

  printf("empty-system wake work: %d\n", r.count);
  check(ok && r.tag == TAG_CASE3 && r.count > 0,
        "process wakes when no RUNNABLE or RUNNING peer exists");

  close(result_pipe[0]);
  close(result_pipe[1]);
  set_nice(0);
}
```

result

```
=== Test 7: wakeup vruntime with no RUNNABLE or RUNNING peer ===
empty-system wake work: 34904
[PASS] process wakes when no RUNNABLE or RUNNING peer exists
```

0x47. test8

```
void
test_sleeping_kill_stability(void)
{
  printf("\n=== Test 8: kill sleeping process stability ===\n");

  int pid = fork();
  if(pid == 0){
    pause(1000);
    exit(0);
  }

  pause(2);
  kill(pid);

  int status;
  wait(&status);
  check(1, "kill of sleeping process does not crash scheduler");
}
```

result

```
=== Test 8: kill sleeping process stability ===
[PASS] kill of sleeping process does not crash scheduler
```

0x48. test 9

```
void
test_many_runnable_stability(void)
{
```

```
  printf("\n=== Test 9: many runnable processes stability ===\n");

  int ok = 1;

  for(int nice = NICE_MIN; nice <= NICE_MAX; nice++){
    int pid = fork();
    if(pid == 0){
      if(set_nice(nice) != 0)
        exit(1);
      count_until(uptime() + 10);
      exit(0);
    }
  }

  wait_all(NICE_COUNT, &ok);
  check(ok, "nice -3 through 2 processes all complete");
  set_nice(0);
}
```

result

